

Project Plan

Delivery objective

Deliver a secure, tested web platform where devotees can discover temples, reserve capacity without overselling, and receive verifiable tickets; organizers can manage only assigned temples; and administrators can manage master data and reporting.

The repository already implements a broad demo feature set. The next plan should prioritize correctness and security rather than adding more UI features.

Work breakdown

Workstream	Current state	Next deliverable
Discovery and UI	Broad demo complete	Accessibility and UX validation
Identity/access	JWT and role middleware present	Remove public privilege selection; route guards; stronger sessions
Temple management	CRUD API; partial admin UI	Complete edit/delete behavior and integrity rules
Slot/booking	Main workflow complete	Atomic capacity, pending cancellation fix, scheduled expiry
Donation	Simulated record/receipt	Decide demo-only label or integrate a real provider safely
Organizer model	Global organizer access	Organizer-temple assignment and scoped queries
Quality	Build/lint only	Unit, integration, concurrency, and E2E test suites
Operations	Local startup only	Environment profiles, containers, CI/CD, monitoring, backup

Recommended milestones

M1 — Security baseline

- Force public registration to `USER` on the server.
- Provision organizers/admins through an authenticated admin flow.
- Require `JWT_SECRET`; remove fallback secret.
- Add schema-based request validation, login/register rate limiting, secure CORS, Helmet, and request size limits.
- Enforce role checks for booking/review behavior on the backend.

M2 — Booking correctness

- Use a conditional atomic update such as `availableSeats >= requestedSeats`.

- Create booking and decrement seats in a transaction when supported.
- Restore seats for both pending and booked cancellation exactly once.
- Run expiry in a background worker; make it idempotent.
- Define cancellation/refund rules and past-slot restrictions.

M3 — Ownership and data integrity

- Add organizer temple assignments.
- Scope slot, booking, and donation queries by assignment.
- Add unique/lookup indexes and validation minima.
- Decide cascade, soft-delete, and record-retention policies.

M4 — Automated quality

- Backend unit tests for controllers/services and state transitions.
- API integration tests with an isolated MongoDB test instance.
- Frontend component tests for validation and route behavior.
- E2E flows for registration, booking, expiry, donation, and administration.
- Concurrency/load tests for the last available seats.

M5 — Deployment readiness

- Environment-driven API/database configuration.
- Separate development, test, staging, and production settings.
- Container images, migrations/index rollout, CI/CD gates, structured logs, monitoring, and backup/restore test.
- TLS, secure headers, secret manager, and production origin allowlist.

Prioritization

Priority	Items
P0 — release blocker	Public admin registration, fallback JWT secret, seat race condition, pending cancellation leak, real-payment ambiguity
P1 — high	Organizer global access, server validation, rate limiting, expiration scheduler, deletion integrity, automated integration tests
P2 — medium	Route guards, lint warnings, accessibility, pagination, audit log, improved errors, environment configuration
P3 — enhancement	Notifications, QR entry validation, richer reporting, localization, media upload

Risks

Risk	Probability	Impact	Mitigation
Unauthorized privilege escalation	High	Critical	Server-owned roles and controlled provisioning

Risk	Probability	Impact	Mitigation
Oversold slots under concurrency	Medium–High	High	Conditional atomic update and transactional workflow
Seats remain unavailable after pending cancellation	High	High	Restore based on prior state; idempotent transition tests
Organizer sees unrelated temple data	High	High	Assignment model and query scoping
Demo payment mistaken for real payment	Medium	High	Prominent labeling or certified provider integration
Orphaned data after deletion	Medium	Medium–High	Soft delete/cascade policy and referential checks
Regression due to no tests	High	High	CI test pyramid and release gates

Definition of done

A production feature is done only when its API authorization and validation are defined, data changes are consistent under concurrency, expected and failure flows are automated, logs contain no secrets or payment data, accessibility is reviewed, documentation is updated, deployment/rollback is tested, and monitoring signals are in place.